Repeat Testing with Mock Data

Medicaid Claims Checker · uploading the same test files round after round · October 20, 2026

The short version. The app accepts each claim only once, on purpose. To test the same mock files again, make a fresh copy whose claim IDs carry a new tag, then upload the copy. One paste does it; use the date and time as the tag so you never have to track numbers.

Why a file is refused the second time

The app refuses a file it has already received, and any claim whose content matches a claim already submitted, even in a different file. This is the production behaviour: the refusal names the earlier batch, appears in Batch History, and emails the admin users. Test data gets no exemption, so an original mock file can be uploaded once per database. **"Clear display" does not reset this**; it only hides old batches.

A copy whose claim IDs end in a new tag (for example `SMP-CLEAN-001-10201545`) is a new claim to the app, so it is accepted and evaluated normally. Nothing else in the claim changes, so the expected results are the same.

Each round: three steps

1. Make copies. In Terminal, from the project folder, paste these lines. It tags every claim with the current date and time and writes the copies to `~/upload_rounds/<tag>`:

```
cd ~/Work/EMRTS/medicaid_claims_checker
T=$(date +%m%d%H%M)
elixir phoenix_web/test/scripts/retag_test_data.exs \
  --tag $T --out ~/upload_rounds/$T \
  mock_data phoenix_web/test/fixtures/x12_suite \
  phoenix_web/test/fixtures/x12 phoenix_web/test/fixtures/json_upload
open ~/upload_rounds/$T
```

It prints one line per file set and opens the new folder in Finder. The later commands use `$T` too: run them in the same Terminal window, or replace `$T` with the folder's name (for example `10201545`). Each folder holds `mock_data`, `x12_suite`, `x12` and `json_upload`, laid out like the originals.

2. Upload. Open `http://localhost:4000/`. In **Manual Upload**, drag in files from the new folder (up to 100 files, 10 MB each) and click **Process Files**. To send a whole set in one go, zip it first, for example:

```
cd ~/upload_rounds/$T/mock_data
zip -qr ../mock_claims.zip claims/clean claims/single_rule \
  claims/multi_rule claims/nppes batches
```

Then upload `mock_claims.zip`. Leave out `manifest.json`, the `.dsl` rule files and the redundancy claims; they are not meant for the upload page.

3. Check results. Open the batch in **Batch History** and compare each claim with the expected result in the `mock_data` README tables or the set's `manifest.json` (claim IDs are the same apart from the tag).

Checking results without clicking

To confirm that every file in a round is judged correctly, run the verify script on that round's copy. It sends each claim through the same steps as an upload and compares the outcome with the manifest, so it can be run any number of times:

```
cd ~/Work/EMRTS/medicaid_claims_checker/phoenix_web
mix run --no-start test/scripts/verify_x12_suite.exs \
  --suite ~/upload_rounds/$T/x12_suite
mix run --no-start test/scripts/verify_x12_suite.exs \
  --suite ~/upload_rounds/$T/mock_data
```

Pass = the last line reads `FAIL 0`. The script needs the engine running; it does not add batches to Batch History.

Testing the duplicate refusal itself

1. Upload any file from the current round. It is evaluated.
2. Upload the same file again. Expect: “Nothing was evaluated: every claim was already submitted. Refused as already submitted: …”, a red “Refused as already submitted” box on that batch in Batch History, and an alert email to each active admin user. When Phoenix runs with the Brevo mail settings (as `./start.sh` does when they are set), the email goes to the admins’ real inboxes, so each test sends real mail. Without those settings it stays in the app, at `http://localhost:4000/dev/mailbox`.
3. Dismiss the message with its **×**, by choosing new files, or with **Clear display**.

Good to know

Situation	What to do
A file from a round is refused	It was already uploaded in that round. Make a new round (step 1); don’t reuse a round’s folder.
X12 error files (truncated, not X12, …)	They hold no claims, so they are never recorded and can be uploaded again as they are.
Redundancy claims don’t match the README	They are written for `redundancy_levels.dsl`; check them with the verify script, not the upload page.
A new provider rule doesn’t fire on an X12 file	Provider tenure comes only from JSON claims; use the `mock_data` JSON files for R8.
Old rounds piling up	`rm -rf ~/upload_rounds/*` deletes the copies (not the originals or anything in the app).
“--tag must be letters and digits”	The tag can only contain letters and digits; the date-and-time tag above always qualifies.